Detecting Prediction Collapse Caused by Pooled Feature Normalization in Space-Trajectory Screening

Harsha Vardan M Sakamuri[1], Rohit Michael Raj[1], Valarmathi P.[1*]

[1]*School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Chennai 600127, India*

*Corresponding author: valarmathi.sudhakar@vit.ac.in

Abstract

Monte Carlo trajectory campaigns spend much of their compute on runs that are sure to fail. A learned screen can stop such runs early, but one common preprocessing choice, a single feature scaler fitted to target groups with very different scales, can break it without warning. Two common checks, a pooled score and a tree baseline on the same inputs, do not reliably show it. This paper proposes OrbitGuard, a framework for learned trajectory screening that scales each target on its own, measures how much signal survives the scaling, and checks the spread of the predictions within each target, without labels at run time. It is evaluated on 70,000 missions to seven planets simulated with NASA's General Mission Analysis Tool, with a Transformer screen and an XGBoost baseline, over three scaling settings, four targets and five seeds (60 runs). With pooled scaling the Transformer collapses on Venus on four of five seeds: the standard deviation of its predictions falls to 3.11×10^{-5} and its test area under the curve (AUC) falls from 0.9998 to 0.5988, while XGBoost on the same inputs keeps a test AUC of 1.0000 and the pooled AUC of the group falls only to 0.875. Bootstrap and McNemar tests confirm the damage on Venus, Mars and Mercury. With a threshold set on validation runs, the check flags the four collapsed runs with no false alarm, and raises none on 30 runs on NASA turbofan engine data. Screens trained on grouped data should therefore be checked within each group.

Keywords: feature normalization, prediction collapse, model diagnostics, deep learning, Monte Carlo simulation, trajectory screening

1 Introduction

Designing a space mission is not done with a single trajectory. Engineers run large Monte Carlo campaigns, in which thousands of trajectories are simulated with small random changes to the launch and the engine burns, to see how often the mission still reaches its target. The Transiting Exoplanet Survey Satellite (TESS) mission of the National Aeronautics and Space

Administration (NASA), for example, used a parallel Monte Carlo campaign of this kind to validate its trajectory and to select it [1]. Every one of these trajectories has to be propagated, which means that a numerical integrator follows the spacecraft forward in time under the gravity of the Sun and the planets. For transfers to the outer planets one run covers years of simulated flight, so a campaign is expensive.

A large share of that cost is wasted. In the campaigns studied here, most of the compute is used on runs that were already sure to fail when they left the parking orbit. Existing work reduces this cost in three ways. The first makes each propagation cheaper, for example by running many samples in parallel on graphics processing units (GPUs) or by expanding the dynamics around a nominal state [2]. The second replaces the Monte Carlo samples with a surrogate model of the uncertainty [3,4], or replaces part of an expensive simulation with a learned model [5]. The third, simpler idea is to watch the first part of each trajectory and stop the runs that are going to fail. A model that makes this decision is called a screen in this paper. Machine learning is a natural fit for it, and learned models are already used widely in astrodynamics [6,7]. These approaches have a common problem. They are judged by their accuracy on held-out data, and a learned model can lose its accuracy through a step of the pipeline that nobody watches.

One such step is feature scaling. Before the data reach the network, every feature is shifted and divided by a typical size, so that features measured in very different units become comparable. This step is old and well understood [8,9], and it is usually done with one line of code. In the data used here, however, the features come from seven different target planets, and their ranges differ by orders of magnitude. If one scaler is fitted to all targets of a group together, the large differences between targets set the scale, and the small differences between missions of the same target, which are the ones that decide success or failure, are squeezed close to zero.

This is what happened in the screen studied here. The model ended up giving a single, almost constant probability of failure to every mission of a target. That a pooled scaler can do this is not surprising. The difficulty is that it is easy to miss. A pooled score shows the damage only in part: in the controlled ablation of Section 5, the area under the receiver operating characteristic curve (AUC) over the three targets of a group falls from 0.9995 to 0.875, while the AUC of the failed target alone is 0.687. A gradient boosted tree trained on the same scaled arrays, which is the usual sanity check, scores almost perfectly, because trees split on

the order of the values and the rescaling does not change that order. Both checks can leave the engineer with the impression that the data are fine and that only the network needs tuning.

This paper proposes OrbitGuard, a framework for learned trajectory screening that guards against this failure. OrbitGuard scales the telemetry of each target on its own, reports how much of the mission to mission signal survives the scaling, and, before the screen is trusted, checks the spread of its predictions within each target. The check needs no labels at run time and no second model. The remedy for the failure, scaling each group separately, is simple, and it is not the contribution. The contribution is a careful description of a failure that a pooled score shows only in part and that the tested tree baseline does not show, together with a check that does show it. The problem is not specific to astrodynamics. Any learned model trained on grouped data, such as engines running under different operating conditions or sensors at different sites, can meet the same failure.

The main contributions of this work are as follows.

- A controlled decomposition of the failure. With the model, the data split and the training fixed, only the scaling statistics change. This shows that pooling over time within one target does little harm, while pooling across targets can make the screen output a constant.
- An analysis of why two usual checks miss the failure. A tree baseline on the same input is not affected by this kind of rescaling, and a pooled score mixes the failed target with the healthy ones.
- The OrbitGuard framework. It keeps per target scaling, reports the signal ratio of the scaled inputs, and adds a label-free check on the spread of the predictions within each target. The check comes with a fully stated threshold protocol, a sensitivity analysis, a comparison with other summaries of the predictions, statistical tests, and checks on batch size and calibration.
- A test on a second, public dataset, the NASA turbofan engine data, in which engines run under six operating conditions that play the role of the targets. There, pooling shrinks the signal about nine times, the model still works, and the check raises no false alarm. An economic analysis then shows where in a campaign a learned screen pays off.

The rest of the paper is arranged in the following manner. Section 2 reviews related work on learned models for trajectory design, guidance and control, orbit prediction, and screening in space operations. Section 3 gives a detailed explanation of the proposed OrbitGuard framework. Section 4 describes the experimental setup, the metrics and the statistical tests.

Section 5 presents the results, the comparison with the usual checks and the statistical analysis. Section 6 discusses what the results mean and what they do not show. Conclusions and future work are given in Section 7.

2 Related work

This section reviews work that is close to the problem of this paper: learned models that replace or shorten an expensive trajectory or orbit computation, and learned models that decide which cases need attention in space operations. Section 2.1 covers learned estimates of the cost and feasibility of transfers. Section 2.2 covers neural guidance and control that is checked in simulation. Section 2.3 covers learned orbit prediction and uncertainty propagation. Section 2.4 covers learned screening of conjunctions and of spacecraft telemetry. For each study we give its aim, its data and training, its main results, and the limitations we observed. Section 2.5 compares the studies in a table, and Section 2.6 states the research gap.

2.1 Learned estimates of transfer cost and feasibility

In preliminary mission design, a very large number of candidate transfers must be evaluated, and a full optimization of each one is slow. A common answer is to optimize a sample of transfers, train a model on them, and use the model as a fast estimate for the rest. A survey by Izzo et al. [6] describes how evolutionary optimization, tree searches and machine learning, including deep learning and reinforcement learning, are applied in spacecraft guidance, dynamics and control, with examples taken from mission design. As a survey it reports no experiments of its own, but it shows how widely learned models are now used in this field.

Shang and Liu [10] assess how easily more than 600,000 main-belt asteroids can be reached. Their training samples come from two transfer models, globally optimal two-impulse transfers and Mars gravity-assist transfers, solved with multistart and deflection techniques to avoid local minima. Gaussian process regression (GPR) then gives the accessibility of the whole population in tens of seconds with an average relative error of 1.33%, and it finds 3,976 candidate targets with a low total velocity change. The study is limited to the two impulsive transfer models that produced the training data.

Zhu and Luo [11] evaluate low-thrust transfers between targets of a multitarget mission. They build a database of both infeasible and optimal transfers and train two multilayer perceptrons (MLPs), one that classifies whether a transfer is feasible and one that estimates its optimal fuel consumption. The classifier reaches an accuracy above 98.3% and the fuel estimate a

relative error below 0.37%, and tests on two asteroid chains show the value of the method for sequence design. Its feasibility classifier is close to a screen, since it decides which transfers deserve a full optimization. The results are reported for the transfer database and the two asteroid chains of the study.

Li, Baoyin and Topputo [12] train neural networks to learn the optimal time, the initial costates and the optimal control law of time-optimal low-thrust interplanetary transfers. The aim is to avoid the difficult choice of a first guess for the costates in indirect optimization. Three networks are trained on a generated dataset. They predict the optimal time and the initial costates accurately, and when they are used to start the shooting function of the indirect method the success rate is 100%. The study covers time-optimal transfers only.

Song and Gong [13] map the orbital characteristics of two orbits to the solar-sail transfer time between them with a deep neural network (DNN). The estimate is then used inside a Monte Carlo tree search to find the best visiting sequence in a multi-asteroid exploration problem, and the sequences found are verified by solving the optimal control problems one after another. Two examples with different backgrounds show that the approach works. The validation rests on these two examples.

Li et al. [14] use deep feed-forward networks for three problems: the transfer time of time-optimal low-thrust transfers, the fuel use of fuel-optimal low-thrust transfers, and the total velocity change of multi-impulse transfers under the J2 perturbation, which comes from the oblateness of the central body. The low-thrust training data are optimized with the indirect method and the multi-impulse data with a J2 homotopy and particle swarm optimization, and the hyperparameters are searched by grid search, random search and tree-structured Parzen estimators. The mean relative error is below 0.5% for low-thrust transfers and below 4% for multi-impulse transfers. The networks are compared with other machine learning methods and tested on cases well outside the training data. The error bound for multi-impulse transfers (below 4%) is much looser than for low-thrust transfers (below 0.5%).

Xie and Dempster [15] address the waste of computing time when low-thrust transfers are generated at random for training data, since most of that time is spent on transfers that turn out to be infeasible. They train a DNN classifier that predicts, before the optimization starts, whether a transfer will be feasible. The classifier reaches an overall accuracy of 97.9%, the best of the algorithms they tested. In its task, this study is the closest to the screening task of

this paper, because a classifier decides which expensive computations to run. It is reported as a preprint, and its result is an overall accuracy.

Yang et al. [16] build a fast solver for the Lambert problem under the J2 perturbation. A DNN corrects the initial velocity given by the unperturbed Lambert solution, and a differential correction step with forward shooting then meets the terminal conditions exactly. Eight forms of the training samples are compared. On a multirevolution test case in the Jupiter system, the solver is much faster than a standard shooting method and a homotopy method at a comparable accuracy. The performance is shown on one representative test case.

Liu, Noomen and Visser [17] build a gravity assist mapping with GPR in a two-dimensional circular restricted three-body model of the Sun, the Earth and Moon, and the spacecraft, with flybys allowed anywhere above 300 km altitude. They study how the number and the distribution of the training samples affect the prediction of the state after the flyby. The trained model is efficient and accurate when compared with numerical integration. The model is limited to the planar circular restricted problem.

Izzo, Sprague and Tailor [18] give a short history of how evolutionary computation and machine learning have been used in interplanetary trajectory design, and then use deep networks to represent the optimal guidance profile of an interplanetary mission on board. In their words, the results are limited to the chosen test case, an Earth to Mars transfer.

2.2 Neural guidance and control checked in simulation

A second group of studies trains networks that output a control action directly from the state of the spacecraft. These networks are trained on optimal trajectories or by reinforcement learning, and they are judged in simulated flights, often in Monte Carlo campaigns. Sánchez-Sánchez and Izzo [19] train deep networks to represent the optimal control action during a pinpoint landing. The landings driven by the trained networks are close to the simulated optimal ones, which supports an onboard real-time optimal controller for a large set of initial states. The study assumes perfect state information.

Cheng et al. [20] propose a real-time optimal controller for landing on asteroids with irregular gravity fields. DNNs first approximate the gravity field from samples computed with a polyhedral model, an approximate indirect method with a homotopy then solves the time-optimal landing problems, and five DNNs learn the relation between the state and the optimal actions. Simulated time-optimal landings on the asteroid Eros show real-time performance, optimality and robustness. The results are for one asteroid.

Izzo and Öztürk [21] show that deep networks can represent the optimal guidance of a mass-optimal low-thrust transfer from the Earth to Venus in a large part of the state space. They propose a method called backward generation of optimal examples, which gives databases with orders of magnitude more optimal trajectories than earlier work, and they train networks for the thrust profile and for the optimal mass. A spacecraft using these networks reaches the target conditions with only 2% more propellant than the optimal transfer, from both nominal and off-nominal starting conditions. The results are for one transfer.

Rubinsztejn, Sood and Laipert [22] train a light neural network by supervised learning on optimal state and action pairs found with direct and indirect methods. The network maps the state of the spacecraft to a near-optimal control action in different dynamical regimes and for different optimality criteria. The solutions keep the optimality and the time of flight of traditional trajectories, and the same networks correct most of the missed thrust events on long low-thrust trajectories. Some missed thrust events are not corrected.

Zavoli and Federici [23] use reinforcement learning for the robust design of low-thrust interplanetary trajectories under process noise, observation noise and errors in the thrust, including missed thrust events. The problem is written as a Markov decision process, and a network is trained with proximal policy optimization to map the observed state to the control. For an Earth to Mars mission, the solution in the unperturbed case is compared with the optimal one from an indirect method, and the robustness of the closed-loop guidance is then assessed with Monte Carlo campaigns. The results are for one mission.

Izzo et al. [7] review end-to-end guidance and control networks for interplanetary transfers, planetary landings and close-proximity operations, and propose drone racing as a test bed on real hardware. As a review it summarizes results of earlier studies.

2.3 Learned orbit prediction and uncertainty propagation

A third group uses learned or reduced models to predict orbits, or to propagate their uncertainty more cheaply than plain Monte Carlo sampling. Peng and Bai [24] train artificial neural networks on historical orbit determination and prediction data of a resident space object in a simulated space catalog. The networks improve the orbit prediction accuracy and generalize both to future epochs and to other objects, and the study also examines the effect of random initialization and of the network structure. The data come from a simulated catalog. The same authors later applied the approach to publicly available data [25]. A survey by Caldas and Soares [26] covers machine learning for orbit determination, orbit prediction

and the modeling of atmospheric density, and notes that physics-based predictions over seven days can be wrong by kilometers.

Balducci, Jones and Doostan [4] propagate orbit uncertainty with separated representations, which write the future state as a sum of products of functions of single inputs. The number of samples needed grows only linearly with the number of uncertain inputs. On three test cases of an Earth orbiting satellite, the method handles up to 20 uncertain inputs and supports a sensitivity analysis of the inputs. The test cases are all Earth orbits.

Massari, Di Lizia and Rasotto [2] compare two ways to propagate nonlinear uncertainty: a Monte Carlo method run in parallel on GPUs, and a method based on differential algebra. They apply both to two-body dynamics and to full n-body dynamics with accurate ephemerides, and compare their accuracy and speed. Both methods make each propagation cheaper and do not decide which propagations to run. Jia and Xin [3] combine a data-driven kriging model with the admissible region of a short observation arc to propagate orbital uncertainty without running Monte Carlo directly, and choose new samples by active learning. On a low Earth orbit and a geosynchronous orbit, the method is more accurate than polynomial chaos and close to Monte Carlo with far fewer samples. The method is shown on these two examples.

2.4 Learned screening in space operations

The last group is the closest in purpose. Here a learned model looks at a very large stream of cases, such as close approaches between objects or telemetry from a spacecraft, and decides which cases need attention. For conjunctions, the European Space Agency (ESA) released a curated dataset of conjunction data messages (CDMs) collected from 2015 to 2019 and ran a competition in which participants predicted the final collision risk between objects. Uriot et al. [27] describe the design and the results of this competition and the lessons it taught about applying machine learning in this domain. Pinto et al. [28] model time series of CDMs with Bayesian recurrent networks of the long short-term memory (LSTM) type. The model predicts all CDM features at once, including when the next message will arrive, together with the uncertainty of each prediction. Acciarini et al. [29] release Kessler, an open source library that trains such networks on collections of CDMs, and add a generative model that produces realistic synthetic CDMs, because real CDM data are sensitive and not publicly available. Both studies depend on CDM data, which are scarce.

Sánchez Fernández-Mellado and Vasile [30] use evidence theory to compute bounds on the probability of collision that take the quality of the data into account, and then train machine learning classifiers to suggest a decision to the operator for each encounter. The classifiers are trained and validated on a virtual database of encounter geometries and uncertainty intervals. The first approach gives very accurate results, and the second, which uses the encounter geometry directly, skips a slow computation step. The authors state that the methods still need improvement and a systematic comparison. Stevenson et al. [31] treat all-vs-all conjunction screening, in which every pair of catalogued objects is checked, as a classification task. They compare input representations and network architectures on a realistic dataset of 170 million object pairs over a seven day screening period, generated with the BAS3E simulation framework of the French space agency (CNES), and benchmark the networks against the classical filters used in operations, both for screening ability and for computing cost. In its setting, this study is the closest to the present paper: a learned screen on simulated astrodynamics data, measured against non-learned baselines.

For telemetry, Hundman et al. [32] detect anomalies with LSTM networks on expert-labeled data from the Soil Moisture Active Passive (SMAP) satellite and the Mars Science Laboratory (MSL) rover Curiosity. They add an unsupervised and nonparametric way to set the alarm threshold, together with ways to reduce false positives learned during a pilot system. Tariq et al. [33] combine a multivariate convolutional LSTM with mixtures of probabilistic principal component analysis for the Korea Multi-Purpose Satellite 2 (KOMPSAT-2). On 22 million telemetry samples collected over ten months, the method is 35.8% better in precision and 18.2% better in F1 score (the harmonic mean of precision and recall) than the best baseline. The evaluation uses one satellite.

Naik, Holmgren and Kenworthy [34] show that traditional machine learning can classify anomalous behavior accurately. On 120 minute windows of telemetry from a scan drive of the Suomi National Polar-orbiting Partnership (NPP) satellite, supervised and semi-supervised methods reach an accuracy of about 99%, with a random forest and a Gaussian elliptic envelope performing best. A model trained on one reaction wheel of the Kepler space telescope warned of high friction in other wheels three to six months before they failed. Only two examples of failure were available. Jiang et al. [35] propose a Transformer and LSTM encoder and decoder for satellite telemetry. With periodic events of the satellite used as anomaly data, it reaches a higher precision than a Transformer or an LSTM alone. The anomalies used in the tests are periodic events rather than real faults.

Recent work also questions how such detectors are evaluated. Herrmann et al. [36] compare deep anomaly detection methods on benchmark datasets and on real satellite telemetry. They show that the reported performance is often overestimated, that simpler methods can beat complex ones, and that the choice of metric matters, and they find that the LSTM method shows promise but has limits in complex systems without a single clear failure channel. To make evaluation fairer, Kotowski et al. [37] built the ESA anomaly detection benchmark. Its dataset holds annotated real telemetry from three ESA missions, two of which are used in the benchmark, and it comes with a hierarchical evaluation pipeline. Typical detectors assessed with it fall short of what operators need, and all parts of the benchmark are publicly available. Ruszczak et al. [38] released a further benchmark from the OPS-SAT CubeSat of ESA, with baseline results from 30 supervised and unsupervised methods, a fixed split into training and test data, and a set of suggested quality metrics.

2.5 Comparison of the related studies

Table 1 compares the most relevant studies by task, data, model and main reported result, and places the proposed OrbitGuard framework next to them. The studies differ in task and in data, but most of them share one pattern: they report how accurate a trained model is on held-out data.

Table 1. Technical comparison of related studies and the proposed OrbitGuard framework.

Study	Task	Data	Model	Main reported result
Shang and Liu [10]	Asteroid accessibility	Over 600,000 asteroids	GPR	Mean relative error 1.33%
Zhu and Luo [11]	Transfer feasibility and fuel	Generated transfer database	MLP	Accuracy above 98.3%
Li et al. [14]	Cost of optimal transfers	Optimized transfers	DNN	Mean relative error below 0.5%
Xie and Dempster [15]	Feasibility before optimization	Random low-thrust transfers	DNN	Accuracy 97.9%
Yang et al. [16]	Perturbed Lambert problem	Jupiter system case	DNN and correction	Faster at similar accuracy
Izzo and Öztürk [21]	Onboard guidance	Earth to Venus transfers	DNN	2% more propellant
Zavoli and Federici [23]	Robust guidance	Earth to Mars, Monte Carlo	Reinforcement learning	Robustness assessed by Monte Carlo
Peng and Bai [24]	Orbit prediction	Simulated catalog	Neural network	Better prediction accuracy
Jia and Xin [3]	Uncertainty propagation	Low and geosynchronous orbits	Kriging	Near Monte Carlo, fewer samples
Uriot et al. [27]	Collision risk	ESA CDMs, 2015 to 2019	Competition models	Lessons for CDM data

Study	Task	Data	Model	Main reported result
Stevenson et al. [31]	Conjunction screening	170 million object pairs	Deep classifiers	Benchmarked against filters
Tariq et al. [33]	Telemetry anomalies	22 million samples	Convolutional LSTM	F1 18.2% above baseline
Naik et al. [34]	Telemetry anomalies	Suomi NPP and Kepler	Random forest	Accuracy about 99%
Herrmann et al. [36]	Detector evaluation	Benchmarks, real telemetry	Deep detectors	Results often overestimated
OrbitGuard (this work)	Early stop of Monte Carlo runs	70,000 missions, turbofan data	Transformer, spread check	4 of 4 collapses flagged

2.6 Research gap

Most of the reviewed studies train learned models on simulated or operational data and judge them by their accuracy, error or precision on held-out data, and the surveys and benchmarks among them are about how such models are built and evaluated. We found no study among them that examines how the preprocessing of the inputs, such as a scaler shared across groups of targets, can make a trained model give a constant output, or that checks for this before the model is used. Nor did we find one that examines a pooled score or a tree baseline on the same input as a detector of this kind of failure. Finally, the learned screens reviewed here decide which transfers to optimize or which conjunctions to analyze. None of them stops individual Monte Carlo trajectory runs early from their own telemetry. The OrbitGuard framework of Section 3 addresses these gaps.

3 The proposed OrbitGuard framework

This section explains the proposed OrbitGuard framework step by step. Section 3.1 gives an overview of the framework. Section 3.2 describes the simulated missions and the telemetry features. Section 3.3 states the screening task in mathematical form. Section 3.4 defines the three feature scaling settings that are compared, and Section 3.5 defines the signal ratio, which measures how much useful variation survives the scaling. Section 3.6 defines the prediction spread check, and Section 3.7 puts all the steps together in one algorithm.

3.1 Overview of OrbitGuard

The framework has five stages, shown in Figure 1. A trajectory simulator first produces the missions and their outcomes. A short window of telemetry from the start of each flight is then scaled, and the scaled window is passed to a learned screen. Finally, before the screen is trusted, its predictions are checked within each target.

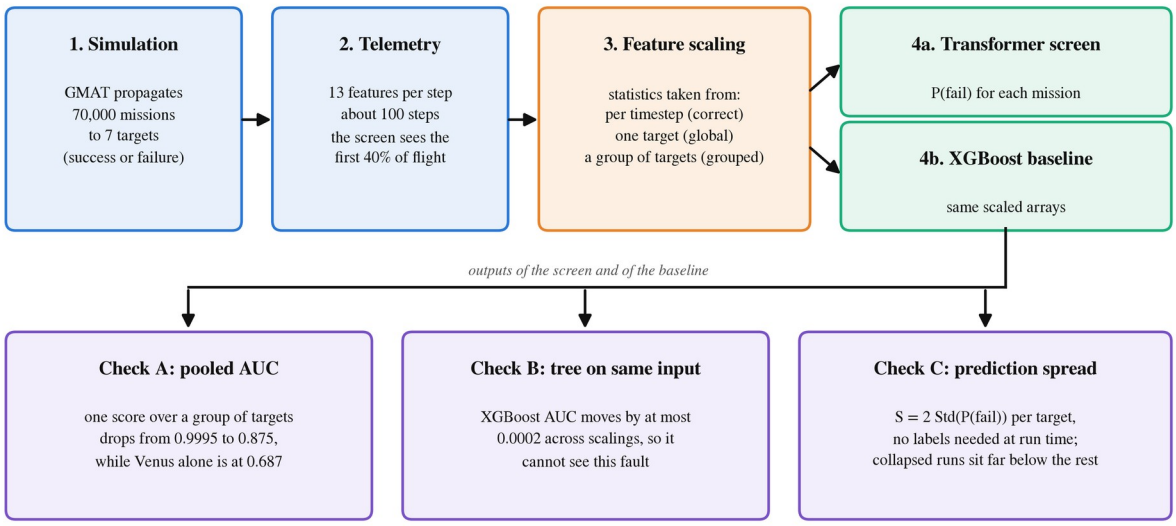


Figure 1. Overview of the proposed OrbitGuard framework and of the two usual checks it is compared with.

Figure 1 reads from left to right along the top row and then along the bottom row. Box 1 is the simulation, in which the General Mission Analysis Tool propagates 70,000 missions to seven targets and labels each one a success or a failure. Box 2 is the telemetry: 13 features at each of about 100 steps, of which the screen sees only the first 40% of the flight. Box 3 is the feature scaling, where the median and the interquartile range, or the mean and the standard deviation, are taken from one timestep, one target, or a group of targets. Boxes 4a and 4b are the two models that read the same scaled arrays, the Transformer screen and the eXtreme Gradient Boosting (XGBoost) baseline. The bottom row holds three checks of the outputs. Checks A and B are the usual ones, a pooled AUC over a group and a tree on the same input, and Section 5 shows that they do not reveal the failure. Check C, the spread of the predictions within each target, is the check that OrbitGuard adds.

3.2 Mission simulation and telemetry data

The screen can only be as good as the data it learns from, so this subsection describes how the missions were simulated and what the screen observes. The data were generated with NASA's General Mission Analysis Tool (GMAT), an open source program for designing and analyzing space missions. Its propagator, the part that integrates the equations of motion forward in time, has been checked against other established mission tools on a large set of test cases [39], and it is deterministic: the same inputs always give the same trajectory.

We simulated 70,000 missions that leave the Earth, 10,000 to each of seven interplanetary targets: Mercury, Venus, Mars, Jupiter, Saturn, Uranus and Neptune. Each mission starts

from a parking orbit around the Earth and then performs a burn that puts the spacecraft on its transfer. Each mission has six injection offsets, which are small random changes to the mission plan. Three are the components of the transfer burn along the velocity, normal and binormal directions (dv_V, dv_N and dv_B). The other three describe the orientation of the parking orbit: the right ascension of the ascending node (RAAN), the argument of periapsis (AOP) and the inclination (INC). Each mission is labeled a success or a failure, and failures are further divided into surface impact, an orbit that is too high, or a missed target. Table 2 gives, for each target, its group, the number of missions, the share of missions that fail, the mean number of telemetry steps per mission after downsampling, and the propagation time of one mission, which is set to 1.2 times the Hohmann transfer time of the target and is the cost that the screen tries to save.

Table 2. The simulated missions of each target.

Target	Group	Missions	Failure share	Steps	Flight time (days)
Mercury	inner	10,000	78.5%	102	127
Venus	inner	10,000	66.2%	94	175
Mars	inner	10,000	73.7%	100	311
Jupiter	outer	10,000	65.1%	101	1,197
Saturn	outer	10,000	65.0%	101	2,650
Uranus	outer	10,000	66.1%	101	7,029
Neptune	outer	10,000	65.0%	100	13,419

The transfers are sampled every 54,000 s (15 hours) and then downsampled to about 100 steps per mission, so that sequences from targets whose flight times differ by two orders of magnitude become comparable. Thirteen features are recorded at each step. Ten describe the motion: the position in a rotating (synodic) frame, the specific orbital energy, the flight path angle, the distance to the target divided by the size of its sphere of influence, the radial velocity toward the target, the speed, the distance from the Earth and the eccentricity. Three are constant for a mission and describe the target: the ratio of its gravitational parameter to that of the Sun, the ratio of its sphere of influence to the transfer distance, and the transfer distance in astronomical units. An eighth target, the Moon, was simulated and deliberately left out. It is a six day transfer near the Earth, sampled every 60 s, with neither a cost structure like that of Section 5.7 nor a heliocentric regime. Figure 2 shows one of the features.

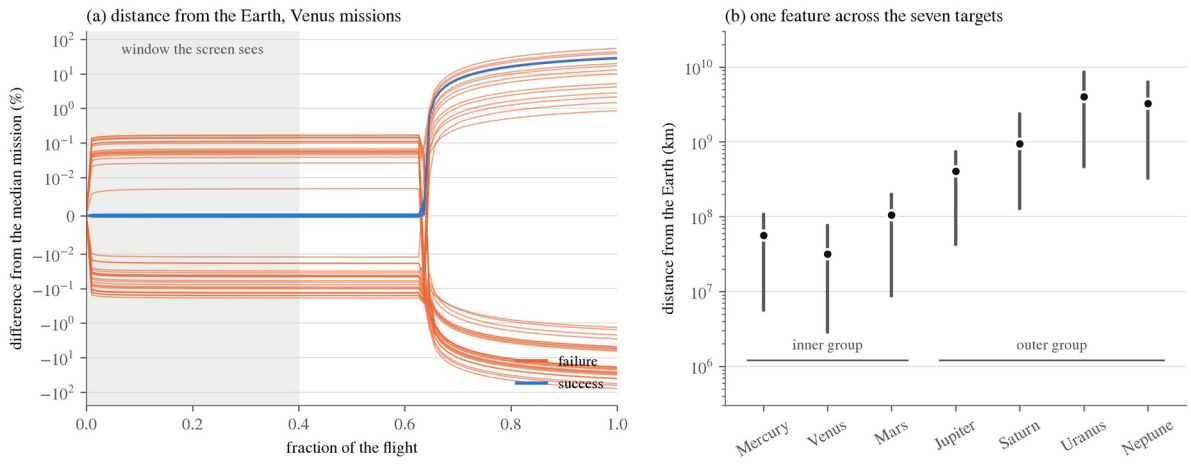


Figure 2. Simulated telemetry. (a) Venus missions inside and after the observed window. (b) Distance from the Earth for each target.

Figure 2 shows why the scaling step matters so much in these data. Panel (a) follows the distance from the Earth along the flight for 40 successful and 40 failed Venus missions drawn at random, written as the percentage difference from the median mission, on a scale that is linear near zero and logarithmic beyond. Over all 10,000 Venus missions, averaged over the window that the screen sees (shaded), 98.6% of the successful missions stay within 0.001% of the median mission. The failed missions differ from it by a median of 0.095%, and 90% of them by between 0.004% and 0.18%. After about 60% of the flight the missions spread apart by one percent or more, but by then most of the propagation cost has been spent. Panel (b) shows the same feature for all seven targets. The median distance runs from about 3×10^7 km for Venus to about 4×10^9 km for Uranus. Within the inner group alone, the middle 90% of the values runs from about 3×10^6 km (Venus) to 2×10^8 km (Mars), almost two orders of magnitude. A scaler fitted to the whole group sets its median and IQR over this pooled range, and the differences of a tenth of a percent that separate the outcomes in panel (a) become tiny next to it.

The missions of each target are split by mission into 70% for training, 15% for validation and 15% for testing. The split is a fixed function of the number of missions and the seed, defined once in the code, so no model is ever scored on data that were used for model selection. This is not a theoretical concern. Two earlier analyses in this project shuffled the same number of missions with the same seed in separate code, and missions that one model had used for validation ended up in the test set of the other analysis.

3.3 The screening task

This subsection states the decision the screen has to make. Mission i of target k has a telemetry sequence $x_{i,t,f}$, where $t = 1, \ldots, L_i$ is the step and $f = 1, \ldots, F$ is the feature, with $F = 13$. Its label is $y_i = 1$ if the mission fails and $y_i = 0$ if it succeeds. The screen observes only the start of the flight, the first T_i steps, with

$$T_i = \lfloor \rho L_i \rfloor \tag{1}$$

where ρ is the observed fraction of the flight, set to $\rho = 0.4$ in this paper, and $\lfloor \cdot \rfloor$ rounds down to a whole number of steps. The screen is a model f_θ with trainable parameters θ that turns the scaled window into a probability of failure:

$$\hat{p}_i = f_\theta\left(z_{i,1}, \ldots, z_{i,T_i}\right) \tag{2}$$

where $z_{i,t}$ is the vector of the 13 scaled features of mission i at step t (Section 3.4) and $\hat{p}_i$, also written $P(\text{fail})$, lies between 0 and 1. The decision is then

$$\text{stop}_i = \mathbb{1}\left[\hat{p}_i \geq \tau_d\right] \tag{3}$$

where $\mathbb{1}[\cdot]$ is 1 when the condition holds and 0 otherwise, and τ_d is a decision threshold chosen on the validation split. Equations (1) to (3) together define the screen. Its value is the propagation time it saves on failed missions, and its cost is the good missions it stops by mistake (Section 4.3).

3.4 Feature scaling settings

Feature scaling is the step whose failure this paper studies, so the three settings are defined exactly here. In every setting, a raw feature value is turned into a scaled value by the rule of Equation (4),

$$z_{i,t,f} = \frac{x_{i,t,f} - m_{t,f}}{q_{t,f}} \tag{4}$$

where $m_{t,f}$ is a center and $q_{t,f}$ is a scale for feature f at step t, both estimated on training missions only. The settings differ only in how m and q are computed and in which part of the data they come from (Figure 3).

The first setting, per timestep scaling, is the one used by OrbitGuard. For each target, feature and step, the center and the scale are the mean and the standard deviation of Equations (5) and (6), taken over the training missions of that target that reach the step:

$$\mu_{t,f} = \frac{1}{|I_t|} \sum_{i \in I_t} x_{i,t,f} \tag{5}$$

$$\sigma_{t,f} = \sqrt{\frac{1}{|I_t|} \sum_{i \in I_t} \left(x_{i,t,f} - \mu_{t,f} \right)^2} \tag{6}$$

where I_t is the set of training missions of the target with $L_i \geq t$ and $|I_t|$ is their number. The scaling then uses $m_{t,f} = \mu_{t,f}$ and $q_{t,f} = \sigma_{t,f}$. A feature that is constant at a step would give $\sigma_{t,f} = 0$, so in that case $\sigma_{t,f}$ is set to 1.

The second and third settings use robust scaling, where the center is a median and the scale is an interquartile range (IQR), the distance between the 25th and the 75th percentile. Robust scaling is a common choice because the median and the IQR are not moved much by outliers. In the global setting, one median and one IQR per feature and target are pooled over all the steps of that target:

$$m_f^{(k)} = \text{median}\left\{ x_{i,t,f} : i \in R_k \right\}, q_f^{(k)} = Q_{75}\left\{ x_{i,t,f} : i \in R_k \right\} - Q_{25}\left\{ x_{i,t,f} : i \in R_k \right\} \tag{7}$$

where R_k is the set of training missions of target k, the sets contain every recorded step $t \leq L_i$ of those missions, and Q_{25} and Q_{75} are the 25th and 75th percentiles. In the grouped setting, the same statistics are pooled over all the steps of all the targets in a group G:

$$m_f^{(G)} = \text{median}\left\{ x_{i,t,f} : i \in R_k, k \in G \right\}, q_f^{(G)} = Q_{75}\{\cdot\} - Q_{25}\{\cdot\} \tag{8}$$

where the sets in $q_f^{(G)}$ are the same as in $m_f^{(G)}$. The inner group is $G = \{\text{Mercury, Venus, Mars}\}$ and the outer group is $G = \{\text{Jupiter, Saturn, Uranus, Neptune}\}$. An IQR of zero is set to 1. In both robust settings, m and q are the same at every step.

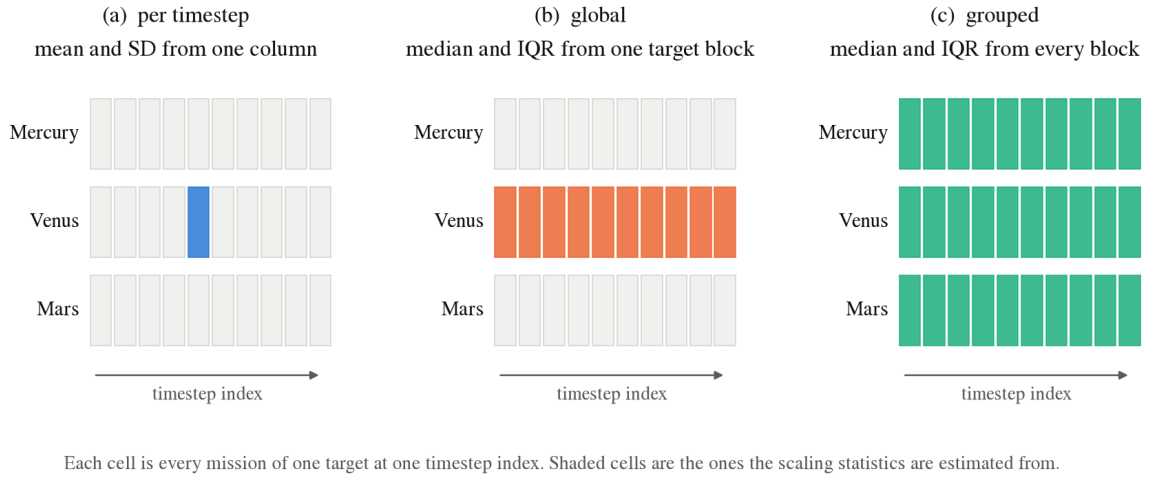


Figure 3. The three scaling settings and the part of the data each one takes its statistics from.

Figure 3 draws the three settings for the three inner targets. Each cell stands for all the missions of one target at one step, and the shaded cells are the ones that the statistics are estimated from. In panel (a), per timestep scaling takes the mean and the standard deviation from a single column of one target, so every step of every target is compared only with itself. In panel (b), global scaling (Equation (7)) pools all the steps of one target, which mixes early and late parts of the flight but keeps the targets apart. In panel (c), grouped scaling (Equation (8)) pools every step of every target in the group, so the median and the IQR are set by the large differences between targets. Global and grouped scaling use the same kind of statistics, so the difference between those two settings is the pooling alone. Everything else in training (the architecture, the seed, the split, the optimizer and the number of epochs) is the same in all three settings.

The grouped setting is not an artificial construction. A single robust scaler fitted to a concatenated training set is what many standard preprocessing tutorials produce. It raises no warning, and the result looks like an ordinary scaled array. It is also the setting that failed in the production system that this study started from.

3.5 Signal ratio

A scaled array can look normal and still have lost the variation between missions that the screen needs. The signal ratio measures how much of that variation survives the scaling. It is the standard deviation of the scaled features across missions, computed at each step and averaged over the features and over the observed window:

$$r = \frac{1}{T-1} \sum_{t=2}^{T} \frac{1}{F} \sum_{f=1}^{F} \mathrm{sd}_i\left(z_{i,t,f}\right) \tag{9}$$

where sd_i is the standard deviation over the missions i of the target that reach step t, $T = \lfloor \rho L \rfloor$ is the observed window with L the longest sequence of the target, and the first step is skipped because every mission starts from nearly the same state. With per timestep scaling, each of the ten motion features has a standard deviation of 1 at every step and the three constant target features scale to 0, so r is about $10/13 \approx 0.77$. A value far below that means that the mission to mission differences have been squeezed toward zero. The window matters. Over the entire flight the measure is dominated by the late steps, where the missions that fail have physically diverged, and it no longer reflects the small early differences that the screen uses. The screen commits at 40% of the flight, so the relevant range is the one inside the window that it observes.

3.6 Prediction spread check

The check that OrbitGuard adds looks at the outputs of the trained screen rather than at its inputs. A collapsed screen gives nearly the same $P(\text{fail})$ to every mission, so the check measures how much the predictions vary within a target. For a target g with a batch of n_g predictions $\hat{p}_1, \ldots, \hat{p}_{n_g}$, the spread is

$$s_g = \sqrt{\frac{1}{n_g} \sum_{i=1}^{n_g} \left(\hat{p}_i - \overline{p}_g \right)^2} \tag{10}$$

where $\overline{p}_g$ is the mean of the n_g predictions. The score reported is

$$S_g = 2\, s_g \tag{11}$$

where the factor 2 makes the score run from 0, for a constant output, to 1, for the widest spread that a probability can have (half of the predictions at 0 and half at 1, which gives $s_g = 0.5$). The score of Equation (11) needs no labels at inference time and no second model. It is computed per target, because pooled over a whole group of targets the spread would mix the levels of the targets and stay high even when one of them has collapsed (Section 5.5). A second score divides by the spread of a healthy model trained on the same data:

$$D_g = \frac{s_g}{s_{\text{healthy}} + \varepsilon} \tag{12}$$

where s_{healthy} is the spread of a model trained with per timestep scaling and $\varepsilon = 10^{-6}$ avoids a division by zero. D_g needs a model that was trained the right way, and when something is broken that is exactly what is not available. OrbitGuard therefore uses S_g as its score, and D_g is reported only for comparison. A target is flagged when its score falls below a threshold:

$$\text{flag}_g = \mathbb{1}\left[S_g < \tau_S \right] \tag{13}$$

where τ_S is the spread threshold. It is set during development, on validation runs whose state is known, as the middle on a log scale of the gap between the collapsed and the working runs:

$$\tau_S = \sqrt{S_c^{\max}\, S_w^{\min}} \tag{14}$$

where $S_c^{\max}$ is the largest validation score of a collapsed run and $S_w^{\min}$ is the smallest validation score of a working run. Labels are therefore used only during development, to say which runs have collapsed and to set τ_S.

The word collapse needs care. Neural collapse describes the geometry of the internal representations and of the classifier in the final phase of training, after the training error vanishes [40]. The collapse studied here is different: the predictions for distinct missions become nearly constant, so the output no longer depends on the input.

3.7 The OrbitGuard algorithm

Algorithm 1 puts the steps of Sections 3.3 to 3.6 together, from the training data to the decision on each mission. Steps 1 to 6 run once during development, and steps 7 to 10 run for every batch of missions in a campaign. In this paper, steps 7 to 10 were evaluated offline, on the saved predictions of the 60 runs (Section 5.5), and not inside a running campaign.

Algorithm 1. The OrbitGuard screening framework

Input: training, validation and campaign missions $x_{i,t,f}$ of each target k; observed fraction ρ; labeled validation runs in known states

Output: a stop or run decision for every campaign mission, or an alarm for a target

1. For each target k, compute the per timestep mean and standard deviation on its training missions (Equations (5) and (6)).
$$\mu_{t,f}, \sigma_{t,f} \leftarrow \mathrm{fit}(x_{i,t,f} : i \in R_k)$$
2. Scale every mission of the target with these statistics (Equation (4)).
$$z_{i,t,f} \leftarrow (x_{i,t,f} - \mu_{t,f}) / \sigma_{t,f}$$
3. Compute the signal ratio of the scaled data (Equation (9)). If a different scaler is ever used, a value far below the 0.77 of per timestep scaling marks that configuration as at risk.
$$r_k \leftarrow \frac{1}{T-1} \sum_{t=2}^{T} \frac{1}{F} \sum_{f} \mathrm{sd}_i(z_{i,t,f})$$
4. Train the screen f_θ on scaled windows of random length, from 5% to 40% of each flight, and keep the epoch with the best validation AUC.
5. Choose the decision threshold τ_d on the validation missions.
6. Set the spread threshold on labeled validation runs (Equation (14)).
$$\tau_S \leftarrow \sqrt{S_c^{\max} S_w^{\min}}$$
7. For each new batch of n_g missions of target g, propagate the first fraction ρ of each flight and score it (Equations (1) and (2)).
$$\hat{p}_i \leftarrow f_\theta(z_{i,1}, \ldots, z_{i,T_i}), T_i = \lfloor \rho L_i \rfloor$$
8. Compute the spread score of the batch (Equations (10) and (11)).
$$S_g \leftarrow 2\mathrm{sd}(\hat{p}_1, \ldots, \hat{p}_{n_g})$$
9. If $S_g < \tau_S$, raise an alarm for target g, set the screen aside, and propagate every mission of the batch in full.
10. Otherwise, stop each mission with $\hat{p}_i \geq \tau_d$ and propagate the rest in full (Equation (3)).
11. **return** the decisions and the alarms

4 Experimental setup

This section describes how the framework was tested. Section 4.1 describes the two models and their training. Section 4.2 describes the 60 runs and the order in which the evaluation choices were made. Section 4.3 defines the evaluation metrics, and Section 4.4 gives the hypotheses and formulas of the statistical tests. Section 4.5 describes the second dataset, and Section 4.6 gives the implementation details.

4.1 Models and training

Two models read the same scaled arrays, a sequence model that serves as the screen and a tree ensemble that serves as the baseline. The screen is a Transformer encoder [41]. A Transformer reads the whole sequence at once and uses attention to let every step look at every other one, which makes it a strong and widely used model for sequences. Each scaled feature vector $z_{i,t}$ is mapped by a linear layer to a vector of width 128 and multiplied by $\sqrt{128}$, a learned summary token, called the classification (CLS) token, is placed at the start of the sequence, and a sinusoidal position code is added to every position, the CLS token included. Each of the 4 layers then applies multi-head attention, in which each of the 8 heads computes the weighted sum of Equation (15),

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q K^T}{\sqrt{d_h}}\right) V \tag{15}$$

where Q, K and V are the queries, keys and values, obtained from the layer input by three learned linear maps, K^T is the transpose of K, $d_h = 128/8 = 16$ is the width of one head, and the softmax turns each row of scores into weights that sum to 1. The layers use pre-layer normalization (Pre-LN), a feed-forward size of 512 and a dropout of 0.1. The output of the CLS token feeds two heads: one predicts whether the mission fails and one predicts the type of failure. The training loss of Equation (16) combines the two heads:

$$\mathscr{L} = \ell_{\text{BCE}} + 0.5\, \ell_{\text{CE}} \tag{16}$$

where ℓ_{BCE} is the binary cross-entropy of the outcome, in which the term of the successful missions, the smaller class, is weighted by the ratio of failed to successful training missions, ℓ_{CE} is the cross-entropy of the failure type, and 0.5 is the weight of the second head. The model is trained on windows of random length, from 5% to 40% of the flight, so that it can give a prediction at any point in the stream and not only at the 40% mark.

The baseline is XGBoost [42], a gradient boosted ensemble of decision trees, with 300 trees of depth 5. It is fitted to the same scaled values, flattened into one row per mission. Every row must have the same length, so the tree reads a fixed window: the first 40% of the shortest mission of the target, which is 14 to 40 steps, with values clipped to ± 10. It is important that the baseline does not work from a separately designed feature set but from the same scaled values that the network sees. This is what the same input means throughout this paper, and it carries most of the argument. Table 3 lists the training settings of both models.

Table 3. Training settings of the models.

Setting	Value
Encoder	Transformer, pre-layer normalization, CLS token
Width, heads, layers	128, 8, 4 (feed-forward size 512, dropout 0.1)
Output heads	failure probability, and failure type (loss weight 0.5)
Loss	binary cross-entropy weighted for class balance
Optimizer	AdamW, learning rate 0.001, weight decay 0.0001
Schedule	cosine annealing, 60 epochs, batch size 128
Gradient clipping	norm 1.0
Training prefixes	random length, 5% to 40% of each flight
Failure-type sampling	weights proportional to $(1/\text{count})^{0.5}$
Model selection	best validation AUC
Baseline	XGBoost, 300 trees, depth 5, learning rate 0.08
Second dataset	same Transformer, 20 epochs, batch size 256, window and horizon of 30 cycles; XGBoost with its default learning rate

4.2 Model states and evaluation protocol

To test the spread check, models are needed in every state, from healthy to collapsed. We therefore repeat the scaling ablation on 5 seeds, 4 targets (Venus, Mars, Mercury and Jupiter) and the 3 scaling settings, which gives 60 runs, and keep the predictions on the validation and test missions of each run. The settings give four states. Per timestep runs are healthy, global runs are mildly hurt, grouped runs that keep working are severely hurt, and grouped runs that cannot rank missions are collapsed. A run counts as collapsed if its AUC on the split in question is 0.85 or lower.

The choices in this evaluation were made in the following order, and we state them plainly because the order matters. The group definitions and the three scaling settings come from the ablation. The AUC cutoff of 0.85 was set after the validation and test AUCs of all 60 runs had been seen. It was placed in a gap between the test AUCs (0.82 to 0.895) and between the validation AUCs (0.81 to 0.907), so it was not chosen in advance, and test results influenced it. Given the cutoff, the spread threshold was chosen on validation predictions and validation AUCs only, with Equation (14). The test predictions were used afterwards, to count the flagged runs. Because of this order, the test result is a check of the threshold and not an independent test of the cutoff, and Section 5.5 reports how the result changes with the cutoff and when the threshold is set on other seeds.

4.3 Evaluation metrics

The metrics measure two things: how well a model ranks missions, and how much a screening decision saves. The main measure of ranking quality is the area under the receiver operating characteristic (ROC) curve, the AUC:

$$\mathrm{AUC}=\frac{1}{n_1 n_0}\sum_{i:\,y_i=1}\sum_{j:\,y_j=0}\left(\mathbb{1}\left[\hat{p}_i>\hat{p}_j\right]+\frac{1}{2}\mathbb{1}\left[\hat{p}_i=\hat{p}_j\right]\right) \tag{17}$$

where n_1 and n_0 are the numbers of failed and successful missions. AUC is the probability that a randomly chosen failed mission receives a higher $P(\text{fail})$ than a randomly chosen successful one, so 0.5 is chance and 1.0 is a perfect ranking. Because AUC depends only on the order of the predictions, it cannot tell a weak model from a constant one on its own, which is why the spread of the predictions is reported as well. For the screening decision of Equation (3),

$$\mathrm{Recall}=\frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FN}},\quad \mathrm{Lost}=\frac{\mathrm{FP}}{\mathrm{FP}+\mathrm{TN}},\quad F_1=\frac{2\,\mathrm{TP}}{2\,\mathrm{TP}+\mathrm{FP}+\mathrm{FN}} \tag{18}$$

where TP is the number of failed missions that are stopped, FN the failed missions that are run, FP the successful missions that are stopped by mistake and TN the successful missions that are run. Recall is the share of failures caught, Lost is the share of good missions lost, and F_1 is the F1 score of the failure class. The economic measure is the share of propagation time saved:

$$\mathrm{Saved}=1-\frac{\sum_k c_k\left(\rho\, n_k^{\mathrm{stop}}+n_k^{\mathrm{run}}\right)}{\sum_k c_k\, n_k} \tag{19}$$

where c_k is the propagation time of one mission of target k in days, n_k is its number of test missions, n_k^{stop} and n_k^{run} are the missions stopped and run in full, and ρ is the fraction of the flight already propagated when a mission is stopped. A stopped mission still costs $\rho=0.4$ of a full run for the telemetry screen and nothing ($\rho=0$) for a screen applied before propagation. Outer planet missions therefore weigh much more than inner ones. For the economic analysis, τ_d is the threshold that catches 99% of the failures on the validation split.

4.4 Statistical tests

Three tests compare per timestep and grouped scaling on the same test missions, all computed from the saved predictions without retraining, at a significance level of $\alpha=0.05$.

Five seeds are few, so the individual runs are reported rather than fitted intervals wherever possible.

The first test is a paired bootstrap of the AUC. The null hypothesis is H_0:Δ=0, with

$$\Delta = \mathrm{AUC}_{\mathrm{pt}} - \mathrm{AUC}_{\mathrm{gr}} \tag{20}$$

where $\mathrm{AUC}_{\mathrm{pt}}$ and $\mathrm{AUC}_{\mathrm{gr}}$ are the test AUCs of per timestep and grouped scaling. The test missions are resampled with replacement $B = 2,000$ times, both AUCs are recomputed on each resample *b*, and the 95% interval of the difference is

$$\mathrm{CI}_{95} = \left[\Delta^*_{(2.5)}, \Delta^*_{(97.5)} \right] \tag{21}$$

where $\Delta^*_{(2.5)}$ and $\Delta^*_{(97.5)}$ are the 2.5th and 97.5th percentiles of the B resampled differences. A resample that contains only one class is skipped. H_0 is rejected when the interval excludes zero.

The second test is McNemar's exact test on the decisions at $P(\text{fail})$ of 0.5. Let b be the number of test missions that the per timestep model gets right and the grouped model gets wrong, and c the number for the reverse. The null hypothesis is that both kinds of disagreement are equally likely, so that b follows a binomial distribution with $n = b + c$ trials and probability 0.5. The two-sided exact p value is

$$p = \min\left(1, 2 \sum_{j=0}^{\min(b,c)} \binom{n}{j} 2^{-n}\right) \tag{22}$$

where $\binom{n}{j}$ is the number of ways to choose j of the n disagreements.

The third test is Welch's t-test on the test AUCs of the two settings across the five seeds, with the null hypothesis that the mean AUCs are equal. The statistic and its degrees of freedom are

$$t = \frac{\overline{A}_{\mathrm{pt}} - \overline{A}_{\mathrm{gr}}}{\sqrt{s^2_{\mathrm{pt}}/n_{\mathrm{pt}} + s^2_{\mathrm{gr}}/n_{\mathrm{gr}}}}, \quad \nu = \frac{\left(s^2_{\mathrm{pt}}/n_{\mathrm{pt}} + s^2_{\mathrm{gr}}/n_{\mathrm{gr}}\right)^2}{\frac{\left(s^2_{\mathrm{pt}}/n_{\mathrm{pt}}\right)^2}{n_{\mathrm{pt}} - 1} + \frac{\left(s^2_{\mathrm{gr}}/n_{\mathrm{gr}}\right)^2}{n_{\mathrm{gr}} - 1}} \tag{23}$$

where $\overline{A}$ is the mean test AUC of a setting over the seeds, s^2 is its sample variance, and $n_{\mathrm{pt}} = n_{\mathrm{gr}} = 5$ are the numbers of seeds. The p value comes from the t distribution with ν degrees of freedom.

Finally, for the share of seeds on which the collapse occurs, the 95% Wilson interval is used:

$$\frac{\hat{\pi} + \frac{z^2}{2m} \pm z\sqrt{\frac{\hat{\pi}(1-\hat{\pi})}{m} + \frac{z^2}{4m^2}}}{1 + \frac{z^2}{m}} \tag{24}$$

where $\hat{\pi}$ is the observed share of collapsed seeds, $m = 5$ is the number of seeds and $z = 1.96$.

4.5 Second dataset

To see what happens outside the GMAT simulator, the framework was also run on the NASA turbofan engine degradation data produced with the Commercial Modular Aero-Propulsion System Simulation (C-MAPSS) [43], a standard public set for remaining useful life (RUL) work [44]. It contains run to failure sensor records for fleets of aircraft engines. Subset FD001 has one operating condition, and FD002 and FD004 have six. The six conditions play the role that the targets play in the GMAT data: the sensors sit at very different levels in each one, and the slow drift that shows an engine wearing out is small next to that gap.

The task is to look at the last 30 cycles of an engine and say whether it will fail within the next 30 cycles. The split is by engine, 70/15/15, so no engine is in two sets. The 14 sensors that change over an engine's life are scaled in two ways: pooled, with one median and IQR for every row, and per group, with one for each operating condition. The model is the Transformer described above with the same optimizer, trained for 20 epochs on 5 seeds, and the baseline is XGBoost on the same scaled window. FD001 has a single condition, so the two scalings are the same there, and it serves as the control. Because engines have no common time base, the signal ratio here is the standard deviation of each scaled sensor within each operating condition over all training rows, averaged over sensors and conditions, and the spread score S is twice the standard deviation of the predictions within each condition, averaged over the conditions.

4.6 Implementation details and use of artificial intelligence tools

This subsection gives the software and hardware used, so that the run times can be judged. All experiments were run with PyTorch 2.13 and XGBoost 3.4.1 on one NVIDIA GeForce RTX 4060 laptop GPU with 8 GB of memory. One run of the ablation takes between about 40 s and 5 min. Every number in this paper is read from machine-readable result files written by the experiment scripts, and the tables and figures are generated from those files.

During the preparation of this work the authors used Claude (Anthropic) to write and debug the experiment code, to draft the text of all sections, and to check the references and the

numbers against the result files. The authors reviewed, edited and verified the content and take full responsibility for it.

5 Results and analysis

This section reports the experiments in the order of the argument. Section 5.1 shows what each scaling setting does to the screen. Section 5.2 shows why the tree baseline does not see the failure, and Section 5.3 repeats the ablation on five seeds. Section 5.4 gives the statistical tests. Section 5.5 evaluates the OrbitGuard spread check, Section 5.6 applies it to the turbofan data, and Section 5.7 asks where in a campaign a learned screen belongs.

5.1 Normalization ablation

The first result is a controlled decomposition of the failure (Table 4). Pooling across time within a single target (the global setting) reduces the signal ratio by up to about 18 times, and the network still works on all four targets, with a loss of a few thousandths of AUC at most. Pooling feature statistics across targets with very different scales does much more damage, and on Venus it gives near-constant predictions: the standard deviation of $P(\text{fail})$ is 3.11×10^{-5}, which is one value for every mission, and the validation AUC (Equation (17)) drops from 0.9999 to 0.6037.

Table 4. Scaling ablation on seed 42. sig: signal ratio; AUC: validation AUC; std: standard deviation of $P(\text{fail})$ on unseen missions.

	per timestep			global			grouped		
Target	**sig**	**AUC**	**std**	**sig**	**AUC**	**std**	**sig**	**AUC**	**std**
Venus	0.7681	0.9999	4.72×10^{-1}	0.0518	0.9947	4.50×10^{-1}	0.0023	0.6037	3.11×10^{-5}
Mars	0.7682	0.9994	4.33×10^{-1}	0.0435	0.9972	4.00×10^{-1}	0.7937	0.9422	4.06×10^{-1}
Mercury	0.7686	0.9994	4.27×10^{-1}	0.1321	0.9970	4.23×10^{-1}	0.0484	0.9069	4.25×10^{-1}
Jupiter	0.7708	1.0000	4.78×10^{-1}	0.0763	1.0000	4.78×10^{-1}	0.0247	0.9997	4.75×10^{-1}

The term collapse, rather than underfitting, is used because of Figure 4. A low AUC alone is ambiguous, since it is also produced by a model that has learned something poor or wrong. A predicted distribution on unseen missions that is three ten-thousandths wide is not ambiguous: the output no longer depends on the input.

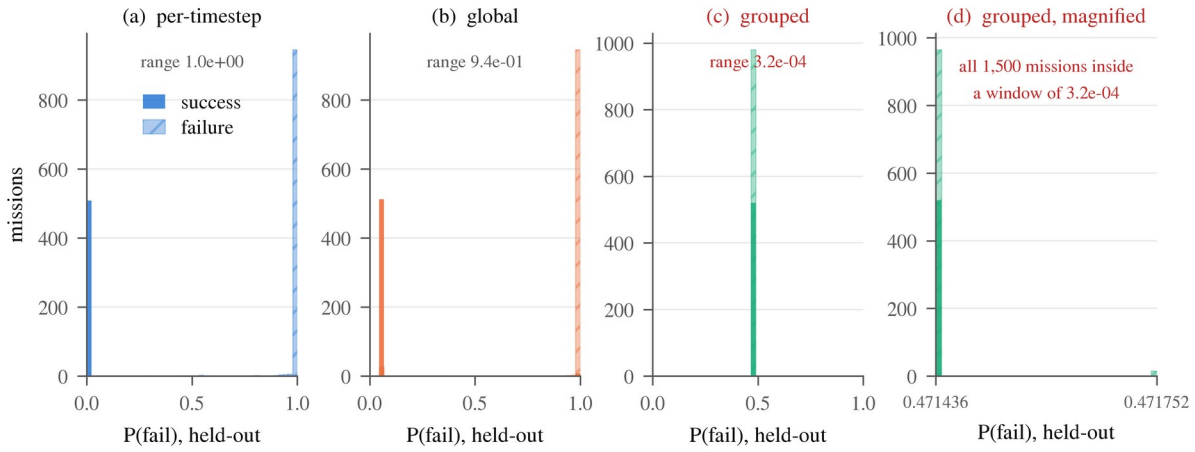


Figure 4. Predicted probability of failure on Venus test missions under each scaling setting.

Figure 4 shows histograms of $P(\text{fail})$ for the 1,500 Venus test missions, with successful and failed missions shown separately. In panel (a), per timestep scaling puts the successful missions near 0 and the failed ones near 1, so the two populations sit at opposite ends of the axis and the range of the predictions is 1.0. Panel (b) shows that global scaling gives almost the same picture, with a range of 0.94. In panel (c), under grouped scaling, every mission receives almost the same value, close to 0.47, so both populations fall into one narrow bar. Panel (d) magnifies that bar and shows that all 1,500 missions lie inside a window of 3.2×10^{-4}. A weak model would still spread its predictions over the axis. This one does not, which is what makes a collapse different from a merely bad model.

The collapse is selective, and the cases that do not break are reported together with it. On the remaining three targets the same manipulation leaves the output varying and costs between 0.0003 and 0.0925 AUC. The signal ratio is not a measure of severity either. Jupiter compresses its signal to 0.0247 and Mercury to 0.0484, yet Jupiter loses 0.0003 AUC while Mercury loses 0.0925. One explanation is that the Jupiter task is easily separable, so that even a very weak signal is sufficient. This was not tested directly. Compression is necessary for the collapse but not sufficient, and how hard the underlying task is appears to modulate it. These data do not support a rule of the form “it collapses when the ratio is below a given value”. The counterexample is visible in Figure 5.

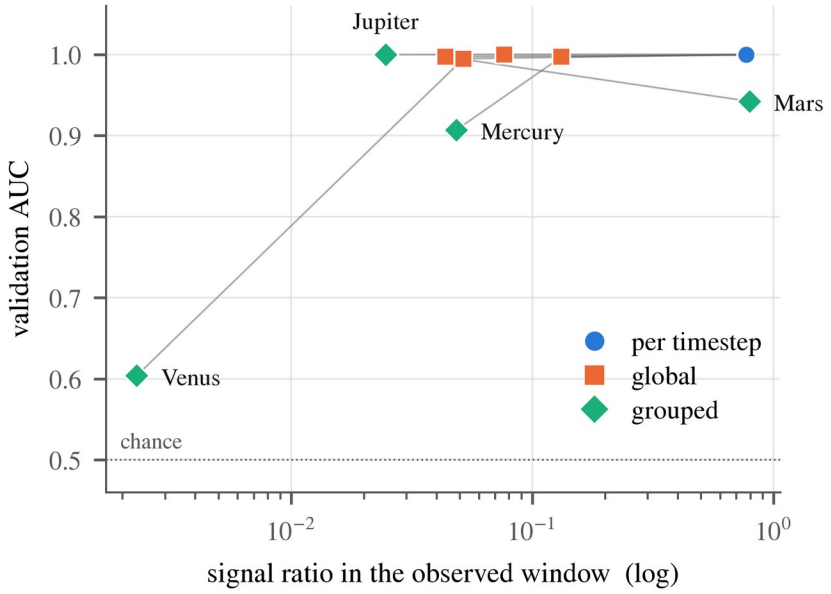


Figure 5. Signal ratio of the scaled inputs against validation AUC.

Figure 5 plots, for each target and scaling setting on seed 42, the signal ratio of Equation (9) on a log scale against the validation AUC. Circles are per timestep runs, squares global runs and diamonds grouped runs, and a thin line joins the points of the same target. The per timestep and global runs all sit near an AUC of 1.0, even though global scaling lowers the signal ratio by up to about 18 times. Among the grouped runs, Venus has the lowest signal ratio, about 0.002, and an AUC of 0.60. Jupiter, however, has a lower signal ratio than Mercury and still keeps an AUC near 1.0, while Mercury drops to about 0.91. If the collapse were caused by compression alone, the points would lie on one curve. They do not, so the signal ratio can warn that a configuration is at risk but cannot predict the outcome.

This ablation isolates normalization and no other part of the model: only the statistics are pooled, and one model is trained for each target. The production failure also shared a single trunk across the group, which makes things worse, since one trunk then has to serve targets whose inputs have all been flattened toward a constant. These numbers therefore probably understate the damage of that configuration, although this was not tested. They show that pooled normalization is enough on its own to destroy a target, not that this is a complete reconstruction of the original incident.

5.2 The tree baseline on the same input

The usual way to check whether a deep model’s poor result comes from the data is to fit a tree ensemble to the same input. Under the grouped setting the XGBoost baseline reports an AUC of 1.0000 on Venus, the target where the network has collapsed to a constant at a

validation AUC of 0.6037, or 0.5988 on the test missions (Table 5 and Figure 6). Its AUC changes by at most 0.0002 across the three settings on all targets, so this scaling choice is not visible to this baseline. The baseline therefore shows that the transformed arrays still contain an ordering that separates the classes. It does not show that the deep model can use the remaining small variation.

Table 5. XGBoost AUC on the same inputs for each scaling setting.

Target	per-timestep	global	grouped	spread
Venus	1.0000	1.0000	1.0000	0.0000
Mars	0.9996	0.9998	0.9998	0.0002
Mercury	0.9997	0.9998	0.9998	0.0001
Jupiter	1.0000	1.0000	1.0000	0.0000



Figure 6. Transformer and XGBoost AUC under grouped scaling on the same inputs.

Figure 6 puts the two models side by side for each target under grouped scaling. The solid bars are the Transformer, with the validation AUC of Table 4, and the hatched bars are XGBoost on the same scaled arrays, with the test AUC of Table 5. The dotted line marks chance. XGBoost reaches 1.000 on every target. The Transformer reaches 1.000 on Jupiter, 0.942 on Mars and 0.907 on Mercury, and only 0.604 on Venus, so on Venus the two models are 0.40 AUC apart. A tree splits on thresholds of single features, and a rescaling that keeps the order of the values does not change which splits are possible. That is why the tree is blind to this failure.

The exact claim matters here. The information is present, and this tree can use it. The comparison does not answer the question that has to be answered, which is whether the network can use the compressed signal. Its invariance to this rescaling makes the check blind

to this particular defect. That statement is deliberately narrower than a claim that tree baselines cannot find preprocessing errors in general.

5.3 Stability across random seeds

A single run could be unlucky, so the whole ablation was repeated with five seeds. Each seed redraws both the train, validation and test partition and the initialization of the network. The compression does not depend on the seed. The signal ratio for Venus under grouped scaling lies between 0.00222 and 0.00232 across all five runs, a spread of about 4.5% against a compression of 335 times. The healthy setting is stable as well: the per timestep AUC of two different seeds differs by at most 0.0010, and Mars, Mercury and Jupiter do not collapse on any seed (Table 6 and Figure 7).

Table 6. Validation AUC over five seeds (mean and standard deviation) and the number of collapsed seeds.

Target	per-timestep AUC	grouped AUC	collapsed
Venus	0.9998 ± 0.0001	0.6912 ± 0.1530	4/5
Mars	0.9992 ± 0.0004	0.9457 ± 0.0093	0/5
Mercury	0.9997 ± 0.0002	0.9118 ± 0.0036	0/5
Jupiter	0.9999 ± 0.0002	0.9997 ± 0.0003	0/5

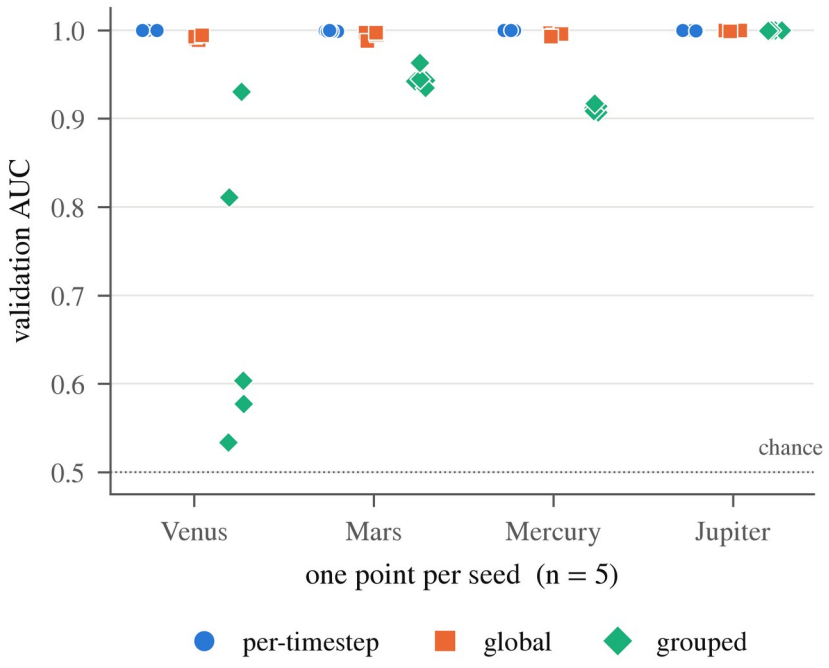


Figure 7. Validation AUC of every run, one point per seed.

Figure 7 shows every one of the 60 runs as its own point instead of a mean with an interval, because with five seeds the raw points are the honest summary. The targets are on the horizontal axis and the validation AUC on the vertical axis, with circles for per timestep,

squares for global and diamonds for grouped scaling. All per timestep and global points sit near 1.0. The grouped points of Mars and Mercury form tight clusters a little lower, near 0.94 and 0.91, and those of Jupiter stay at 1.0. The grouped points of Venus are spread from 0.53 to 0.93, which shows that whether the network escapes the collapse depends on the seed.

The standard deviation of the Venus predictions falls below 10^{-3} on 4 of 5 seeds, to between 1.57×10^{-5} and 4.84×10^{-5} on unseen missions, four orders of magnitude below the 4.66×10^{-1} of correct normalization. On the remaining seed it does not collapse, and it does not recover either: its standard deviation of 3.64×10^{-2} is still an order of magnitude short, at an AUC of 0.9305 against a healthy 0.9999. The observed recurrence of 4 out of 5 has a 95% Wilson interval (Equation (24)) of 0.376 to 0.964. That wide interval describes how often the collapse recurs under these seeds. It is not an interval for the sensitivity of a detector, and it does not support universal reliability.

This seed to seed variation is the main reason for the spread check. The grouped AUC of Venus ranges from 0.5338 to 0.9305 across these runs, which overlaps the AUC of a merely weakened target: Mercury lies inside that range on every seed while still telling missions apart. AUC alone therefore cannot separate the two states, while the prediction spread can. The invariance of the baseline is also repeated. Across 5 seeds, 4 targets and 3 settings, which is 60 fitted XGBoost models, the AUC stays between 0.9989 and 1.0000, and within each seed and target it changes by at most 0.0003 between settings.

5.4 Statistical significance

The tests of Section 4.4 check whether the damage from grouped scaling is larger than chance. Table 7 gives, for seed 42, the AUC of each setting with its 95% bootstrap interval (Equation (21)), the difference with its interval, the McNemar counts *b* and *c* with the exact p value (Equation (22)), and the Welch statistic and p value across the five seeds (Equation (23)). On seed 42 the bootstrap interval of the AUC difference Δ of Equation (20) excludes zero for Venus, Mars and Mercury, and McNemar's test shows that the grouped model gets a different set of missions wrong on those three targets. For Venus, the grouped model is wrong on 974 test missions that the per timestep model gets right, against 9 the other way. For Jupiter the difference is small, and its interval touches zero.

Table 7. Statistical tests of per timestep against grouped scaling on the same test missions.

Target	Per timestep AUC	Grouped AUC	Difference	McNemar	Welch
Venus	0.9998 (0.9996 to 0.9999)	0.5988 (0.5685 to 0.6280)	0.4010 (0.3717 to 0.4313)	974/9, <0.001	4.05, 0.015
Mars	0.9997 (0.9992 to 0.9999)	0.9418 (0.9306 to 0.9528)	0.0578 (0.0468 to 0.0691)	157/9, <0.001	10.74, <0.001
Mercury	0.9993 (0.9987 to 0.9998)	0.9130 (0.8989 to 0.9264)	0.0863 (0.0730 to 0.1002)	191/1, <0.001	21.97, <0.001
Jupiter	1.0000 (1.0000 to 1.0000)	0.9990 (0.9977 to 1.0000)	0.0010 (0.0000 to 0.0023)	5/0, 0.062	0.04, 0.971

Over all 20 pairs of target and seed, the bootstrap interval of the AUC difference excludes zero in 16 and McNemar's test is significant in 16. The two tests agree on 18 pairs (both significant on 15, and neither on 3 Jupiter pairs), and the two pairs where they disagree are also Jupiter runs. Across the five seeds, Welch's t-test separates the two settings for Mars, Mercury and Venus, and not for Jupiter. These tests confirm that the damage is real and not a chance result of one split. With five runs per setting, however, they say nothing about how often such a collapse would occur in other pipelines.

5.5 Evaluating the prediction spread check

This subsection tests whether the OrbitGuard spread check finds the collapsed runs and leaves the others alone. The four collapsed runs are exactly the four that fall below the earlier, stricter rule of a standard deviation of the predictions under 10^{-3}. At the AUC cutoff of 0.85, the spread threshold chosen on validation with Equation (14) is 2.7×10^{-3}, in the middle of the gap between the largest S of a collapsed run (1.0×10^{-4}) and the smallest S of a working run (0.071). Applied unchanged to the test predictions with the rule of Equation (13), it flags 4 runs out of 60, which are the 4 with a test AUC of 0.85 or lower, and no working run (Table 8 and Figure 8). The score D of Equation (12), with its threshold chosen the same way, flags the same 4, so the reference model did not add anything here.

Table 8. The 60 runs grouped by state, with the spread score S and the median AUC on the test missions.

State	Runs	S (smallest to largest)	Median AUC
Healthy	20	0.807 to 0.963	0.9996
Mild	20	0.748 to 0.962	0.9955
Severe	16	0.073 to 0.961	0.9430
Collapsed	4	3.15×10^{-5} to 9.69×10^{-5}	0.5912

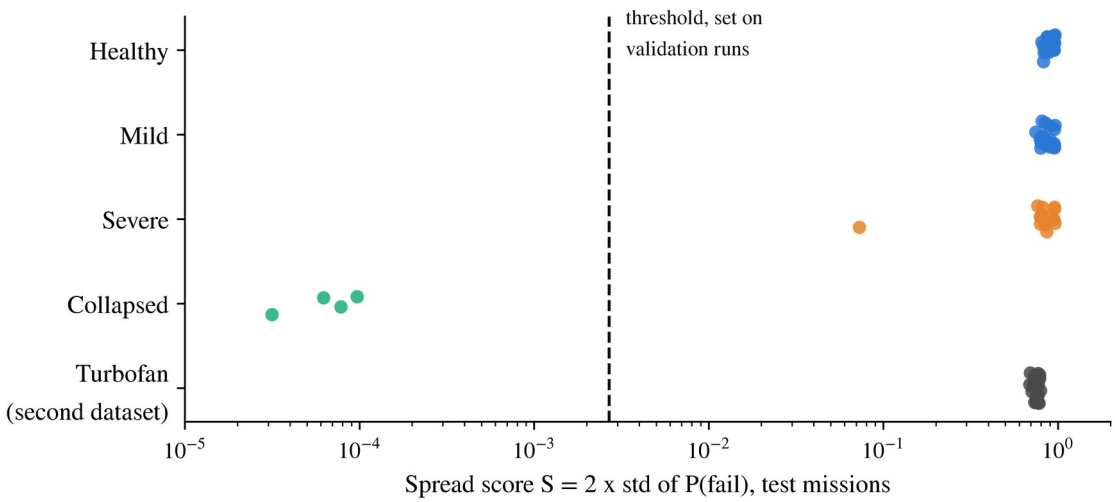


Figure 8. Spread score S of every run on the test missions, with the threshold set on validation.

Figure 8 plots the spread score S of every run on the test missions on a log scale, one row per state, with the turbofan runs of Section 5.6 in the bottom row. The dashed line is the threshold of 2.7×10^{-3} set on the validation runs. The healthy and mildly hurt runs all lie near 1, and so do the severely hurt runs, except one Venus run near 0.07. The four collapsed runs lie between about 3×10^{-5} and 1×10^{-4}, far to the left of the threshold, more than 750 times below any other run. The turbofan runs lie near 0.75, and none of them is flagged. The gap between the two sides of the line is wide, so the result does not depend on the exact position of the threshold inside the gap.

The score is meant for collapse, not for every loss of accuracy. The Mercury runs under grouped scaling lose about 0.09 AUC while their spread score stays above 0.75, so the check does not flag them. Venus on seed 3 is the hard case. Its S is 0.073, about ten times below a healthy run, and it still ranks missions at an AUC of 0.92. It is above the threshold and is not flagged.

Table 9 shows how the result depends on the AUC cutoff. A single threshold can be set on the validation runs only at cutoffs of 0.85 and 0.90. Below 0.85 the Venus run on seed 2, with a validation AUC of 0.81, is labeled as working, but its spread (7.9×10^{-5}) lies inside the range of the collapsed runs, so no threshold separates the two labeled groups. At 0.95 fourteen runs count as collapsed, including Mercury runs that keep their spread, and the groups overlap again. The ranking is more stable than the threshold: the ROC AUC of S on the test predictions is 1.000 for every cutoff from 0.60 to 0.85 and 0.993 at 0.90. The score separates runs with a near-constant output from the others. Whether a run with a constant

output but a usable ranking, such as Venus on seed 2, is called collapsed depends on the cutoff.

Table 9. Effect of the AUC cutoff that defines a collapsed run.

Cutoff	Collapsed (val/test)	Threshold	Missed	False alarms	S	IQR	Range	Entropy
0.60	2/3	none	n/a	n/a	1.000	0.988	0.994	0.982
0.70	3/3	none	n/a	n/a	1.000	0.988	0.994	0.982
0.80	3/3	none	n/a	n/a	1.000	0.988	0.994	0.982
0.85	4/4	2.7×10^{-3}	0	0	1.000	1.000	1.000	1.000
0.90	4/5	2.7×10^{-3}	1	0	0.993	0.945	0.978	0.989
0.95	14/14	none	n/a	n/a	0.879	0.787	0.984	0.972

In Table 9, the second column counts the collapsed runs on validation and on test, the threshold column gives the threshold set on validation (none when the labeled groups overlap), and the missed runs and false alarms are counted on test. The last four columns give the ROC AUC, on test, of four label-free summaries of the predictions. To test the threshold on seeds it was not chosen on, we set it on the validation predictions of four seeds and applied it to the test predictions of the fifth, for each seed in turn. At a cutoff of 0.85 this gives 4 collapsed runs, 0 missed and 0 false alarms over the 60 runs. At 0.90 there are 5 collapsed runs, 1 missed (a Mercury run that keeps its spread) and 0 false alarms. At the cutoffs from 0.65 to 0.80, no threshold could be set on three of the seeds, for the reason given above, and on the other two there was 1 missed run and 1 false alarm. These seeds redraw the split and the initialization, but they use the same four targets, so this is not a test on new targets.

The statistic is an ordinary output variance, and we do not claim that it is the best summary. Table 9 also gives the ROC AUC of three other label-free summaries of the same predictions. Up to a cutoff of 0.85 all of them separate the collapsed runs well (0.98 to 1.00). At 0.90 the spread score is the highest (0.993, against 0.989 for the entropy, 0.978 for the range and 0.945 for the IQR), and at 0.95 the range and the entropy are higher than it. The differences are small and rest on four collapsed runs from one target. What these results support is that a summary of the predictions computed per group detects a failure that the tested baseline does not detect and that a pooled score shows only in part. They do not support a claim that this score is better than other output summaries. We also tried a fourth summary, the share of predictions close to the median. It did as well as the others up to 0.85, but fell to 0.818 at 0.90 and 0.293 at 0.95, so we do not use it.

A pooled score shows the collapse only in part. For the inner group (Mercury, Venus and Mars) the AUC over the test missions of all three targets is 0.9995 with per timestep scaling and 0.875 with grouped scaling, averaged over the five seeds. That is a visible drop. It is, however, 0.19 above the AUC of Venus alone (0.687) and above the mean of the three per target AUCs (0.845), because a model that is constant within a group still ranks the groups against each other. The pooled value also does not say which target failed. The spread score S is computed per target, and it is 3.1×10^{-5} to 9.7×10^{-5} for the four collapsed Venus runs, against 0.7 or more for Mercury and Mars under grouped scaling. Computed over the test missions of the whole inner group instead, S stays between 0.67 and 0.70 on the four collapsed seeds, far above the threshold, so the check has to be run per target.

The spread is computed over a batch of predictions, so it depends on the batch size and on how the probabilities are calibrated. We resampled the saved test predictions at random. Batches of 30 or more missions flagged every collapsed run and no working run. Batches of 10 flagged 1.1% of the working batches, and batches of 5 flagged 7.4%. These batches contain both outcomes, because they were drawn at random from the test missions. A batch that holds missions of one outcome only, or from one narrow region of the inputs, was not tested, and a small spread would be expected there. We also rescaled the predicted log-odds by a temperature between 0.25 and 4. The smallest spread of a working run then moved between 0.28 and 0.018, and the threshold of 2.7×10^{-3} still separated the 4 collapsed runs from the 56 working runs. This is a monotone change of the outputs of one model family. A model with different calibration, architecture, label prevalence or batch size needs its own threshold.

The signal ratio of Equation (9) can be measured before any training, so it can serve as an early warning, as in step 3 of Algorithm 1. Every collapsed run has a signal ratio of 0.0022 or 0.0023. Five runs are below 0.01, namely the 4 collapsed runs and Venus on seed 3, and no run at 0.0243 or above collapsed. The ratio can therefore warn that a configuration is at risk, but it cannot say which of the runs will collapse. For that, the spread after training is needed.

5.6 Second dataset

The turbofan data test whether the check stays quiet on a different kind of data where pooling also shrinks the signal. On these data, pooling shrinks the signal about nine times, from 0.802 to 0.083 on FD002 and from 0.859 to 0.097 on FD004. The model does not break (Table 10). On both subsets the pooled Transformer is slightly ahead of the per group one, the spread of

its predictions is the same, and the tree performs well either way. The spread check, with the threshold it already had, flags none of the 30 runs.

Table 10. Results on the turbofan data over five seeds. signal: signal ratio; AUC on test engines (mean and standard deviation); S: spread score.

Subset	Scaling	signal	Transformer AUC	Tree AUC	S
FD001	pooled	0.784	0.9916 ± 0.0017	0.9953	0.760
FD001	per-group	0.784	0.9916 ± 0.0017	0.9953	0.760
FD002	pooled	0.083	0.9898 ± 0.0014	0.9862	0.767
FD002	per-group	0.802	0.9855 ± 0.0025	0.9925	0.755
FD004	pooled	0.097	0.9910 ± 0.0010	0.9890	0.734
FD004	per-group	0.859	0.9881 ± 0.0044	0.9933	0.731

This agrees with the ablation. Shrinking the signal is not enough to break a model: in the GMAT data Mercury and Jupiter lose most of their signal under grouped scaling and keep working, and here the drift is still present after pooling and the model finds it. It also shows that the check does not raise an alarm on a second kind of data. It does not show that the threshold carries over to other domains.

5.7 Where the screen belongs

The final question is whether a sequence model is the right tool for this screen at all. A classifier that uses only the six injection offsets, an XGBoost model applied before any propagation (called T0), saves 64.5% of the compute, against 38.3% for the telemetry Transformer at 40% of the flight (called T40), although it loses more good missions, 0.76% against 0.21% (Table 11). The compute saved is computed with Equation (19), and the good missions lost and the failure recall with Equation (18). Accuracy also levels off between 10% and 40% of the flight. This is not a modeling fault. The simulator is deterministic, so the outcome is a fixed function of the injection offsets, and the trajectory is an integral of those offsets. It cannot carry information that the offsets do not have.

Table 11. Compute saved by each screen on the test split, with thresholds set on validation.

Screen	Compute saved	Good missions lost	Failure recall
T0: six launch parameters, before propagating	64.5%	0.76%	99.06%
T40: telemetry Transformer at 40%	38.3%	0.21%	98.87%
Cascade: T0 where confident, else T40	65.0%	1.62%	99.94%

This does not make the task trivial. Logistic regression on the same six offsets reaches only 0.4824 to 0.5410 AUC on the seven targets, which is chance, so the mapping from the offsets to the outcome is highly nonlinear and needs a learned model. It does not need a temporal

one. A sequential screen is worth using only when the outcome is not fixed at injection: with stochastic effects during flight, unmodeled dynamics, sensor noise, or campaigns where the launch parameters are unknown. That is the setting that reinforcement learning trajectory work faces when it certifies policies through dispersed Monte Carlo campaigns [23]. For the T0 screen, a five-split analysis (seeds 0 to 4) gives compute saved of 64.50% ± 0.61 percentage points, good missions lost of 1.24% ± 0.24, and failure recall of 98.99% ± 0.18 (mean and standard deviation across five splits). The T40 and cascade rows remain single seed estimates. Five correlated resplits of one synthetic corpus do not measure uncertainty across campaigns or domains.

6 Discussion

This section separates what the experiments show from what they suggest, and lists the limits of the study. Section 6.1 discusses what the results mean for mission assurance, and Section 6.2 gives the limitations.

6.1 What the results mean for mission assurance

The first paragraph below states what the experiments in this paper show. The second and third describe consequences and a workflow that have not been tested.

Demonstrated result. Within the GMAT experiment, the per target spread of the predictions exposes four near-constant Venus runs that the tested XGBoost comparison does not identify and that a pooled AUC shows only in part. Because it checks the behavior at the interface between preprocessing and model, the same kind of check can support verification and validation in other simulation screening pipelines: it tests whether changing the candidate inputs still changes the output of the learned screen. It does not validate the trajectory propagator, prove physical correctness, certify flight software, or establish a threshold outside the evaluated configuration.

Operational implication. If a similar collapse went unnoticed in a mission design campaign, the screen could assign nearly the same decision to very different candidates. Depending on that constant and on the decision threshold, it could discard viable trajectories, keep expensive failures, distort estimates of the size of the feasible region, and bias trade studies or robustness margins. A healthy score from other target groups could hide the affected group and create false confidence in the campaign. These consequences are analyses of possible failures, not outcomes measured in a flight campaign.

Proposed pipeline. During development, a team using OrbitGuard would generate labeled working, degraded and deliberately broken validation states for each combination of dataset, architecture, target group and preprocessing version. It would set a spread threshold for each target and a minimum batch size on those states, freeze both before the untouched test evaluation, and record false alarm and miss rates with their uncertainty. At run time, each screening batch would log the versions of the model, scaler, data and groups, compute the spread of each target alongside input-signal and feature-shift checks, and compare only with the matching frozen limits. An alarm would set the learned screen aside, keep all candidates, and fall back to full propagation rather than stopping trajectories, as in step 9 of Algorithm 1. A tree comparison can help offline diagnosis by asking whether information remains in the same arrays, but it is not needed to compute the run-time statistic.

6.2 Limitations

The results have the following limits, each of which points to further work.

- Seeds. Replication is limited to 5 seeds. The observed recurrence of 4 out of 5 has a wide 95% Wilson interval (0.376 to 0.964), and five seeds rule out neither an unknown controlling factor nor substantial uncertainty.
- Threshold. The AUC cutoff that defines a collapsed run (0.85) was set after the validation and test AUCs had been seen. The spread threshold (2.7×10^{-3}) was chosen on validation predictions only, so the test result is a check of that threshold and not an independent test of the cutoff. A single threshold can be set only for cutoffs of 0.85 and 0.90. All 4 collapsed runs are Venus, so one kind of collapse has been tested. The threshold is not expected to transfer unchanged to another dataset, architecture, calibration, target group, prevalence or batch size, and batches with one outcome class were not tested.
- Datasets. The collapse is shown on the GMAT data. The turbofan data show the same loss of signal without a collapse and without an alarm. That supports the absence of false alarms there, but it does not show that the threshold carries over to other domains.
- Targets. One of four targets collapses, and all collapsed runs are Venus. Other target combinations and group definitions are needed to estimate how much the composition of a group matters.
- Architecture. An LSTM control on the ordinary, correctly preprocessed screening task reaches a test AUC of 0.9928 against 0.9958 for the Transformer, so the base task is not unique to one architecture. The LSTM was not run under grouped scaling, so whether the collapse occurs beyond the Transformer remains open.

- Severity. The check flags near-constant output. It does not flag a model that loses accuracy but keeps its spread, such as Mercury, and graded compression sweeps were not built.
- Synthetic and deterministic data. There are no execution errors, no unmodeled accelerations and no sensor noise, and all missions of a target share a common time base. That common time base is what makes per timestep scaling possible, and it is a property of the generator rather than of a real campaign. Public satellite telemetry benchmarks [32,37,38] are a natural next place to test the check.

7 Conclusion and future work

This paper studied a silent failure of learned trajectory screens and proposed the OrbitGuard framework to detect it. When one scaler is pooled across groups with different scales, the tested Transformer can turn into a constant for one group, while the tested XGBoost baseline, which is not affected by this rescaling, still finds the same arrays separable and a pooled score shows the damage only in part. None of the pieces here are unusual. The defect enters through a routine preprocessing step, the model is a plain Transformer, and the baseline is the one most practitioners would choose. The conclusion is not that trees cannot diagnose preprocessing errors, but that a check which is invariant to this transformation cannot answer whether a scale-sensitive network can use the compressed signal.

The fix, normalizing each group separately, is not new. The lesson is about detection: monitor the behavior of the outputs within each group, in addition to pooled scores and baselines, as OrbitGuard does. Computing the prediction spread needs no labels at run time, but turning it into an alarm needs a threshold set for each model version and an independent labeled evaluation during development. On the GMAT data, the check flagged all four collapsed runs with no false alarm. On a second dataset, where pooling shrank the signal and the model survived, the same check raised no alarm.

Future work should test the check on real telemetry, on other architectures such as LSTMs trained under grouped scaling, on more target groups and group definitions, and on graded levels of signal compression, with the threshold fixed before any test data are seen. For trajectory screening in particular, a screen based on the injection parameters should be preferred whenever the simulator is deterministic, and a sequential screen should be reserved for campaigns in which the outcome is not fixed at injection.

Data availability

The simulated mission data were generated with NASA's General Mission Analysis Tool. The per target extracts used in the experiments can be regenerated with the scripts in the code repository and are available from the corresponding author on reasonable request. The full mission table (about 71 GB) is too large to share in a public repository. The NASA C-MAPSS turbofan engine degradation data are publicly available from the NASA Prognostics Center of Excellence data set repository.

Code availability

The code for all experiments, the result files from which every number is read, and the code that produces the tables and figures are available at https://github.com/Haise-727/gmat-pred.

References

[1] Nickel, C., Dichmann, D. J., Lebois, R., Lutz, S. & Parker, J. J. K. Monte Carlo Analysis as a Trajectory Design Driver for the TESS Mission. In *AIAA/AAS Astrodynamics Specialist Conference* (2016). https://doi.org/10.2514/6.2016-5665

[2] Massari, M., Di Lizia, P. & Rasotto, M. Nonlinear Uncertainty Propagation in Astrodynamics Using Differential Algebra and Graphics Processing Units. *Journal of Aerospace Information Systems* **14**, 493–503 (2017). https://doi.org/10.2514/1.I010535

[3] Jia, B. & Xin, M. Data-Driven Surrogate Model and Admissible-Region-Based Orbital Uncertainty Propagation. *Journal of Aerospace Information Systems* **19**, 753–770 (2022). https://doi.org/10.2514/1.I011107

[4] Balducci, M., Jones, B. & Doostan, A. Orbit uncertainty propagation and sensitivity analysis with separated representations. *Celestial Mechanics and Dynamical Astronomy* **129**, 105–136 (2017). https://doi.org/10.1007/s10569-017-9767-7

[5] Yin, J., Wang, F. & Shankar, M. Strategies for Integrating Deep Learning Surrogate Models with HPC Simulation Applications. In *IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)* (2022). https://doi.org/10.1109/IPDPSW55747.2022.00222

[6] Izzo, D., Märtens, M. & Pan, B. A survey on artificial intelligence trends in spacecraft guidance dynamics and control. *Astrodynamics* **3**, 287–299 (2019). https://doi.org/10.1007/s42064-018-0053-6

[7] Izzo, D. et al. Optimality principles in spacecraft neural guidance and control. *Science Robotics* **9**, eadi6421 (2024). https://doi.org/10.1126/scirobotics.adi6421

[8] Sola, J. & Sevilla, J. Importance of input data normalization for the application of neural networks to complex industrial problems. *IEEE Transactions on Nuclear Science* **44**, 1464–1468 (1997). https://doi.org/10.1109/23.589532

[9] Huang, L. et al. Normalization Techniques in Training DNNs: Methodology, Analysis and Application. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **45**, 10173–10196 (2023). https://doi.org/10.1109/TPAMI.2023.3250241

[10] Shang, H. & Liu, Y. Assessing Accessibility of Main-Belt Asteroids Based on Gaussian Process Regression. *Journal of Guidance, Control, and Dynamics* **40**, 1144–1154 (2017). https://doi.org/10.2514/1.G000576

[11] Zhu, Y.-H. & Luo, Y.-Z. Fast Evaluation of Low-Thrust Transfers via Multilayer Perceptions. *Journal of Guidance, Control, and Dynamics* **42**, 2627–2637 (2019). https://doi.org/10.2514/1.G004080

[12] Li, H., Baoyin, H. & Topputo, F. Neural Networks in Time-Optimal Low-Thrust Interplanetary Transfers. *IEEE Access* **7**, 156413–156419 (2019). https://doi.org/10.1109/ACCESS.2019.2946657

[13] Song, Y. & Gong, S. Solar-sail trajectory design for multiple near-Earth asteroid exploration based on deep neural networks. *Aerospace Science and Technology* **91**, 28–40 (2019). https://doi.org/10.1016/j.ast.2019.04.056

[14] Li, H., Chen, S., Izzo, D. & Baoyin, H. Deep networks as approximators of optimal low-thrust and multi-impulse cost in multitarget missions. *Acta Astronautica* **166**, 469–481 (2020). https://doi.org/10.1016/j.actaastro.2019.09.023

[15] Xie, R. & Dempster, A. G. Feasible Low-thrust Trajectory Identification via a Deep Neural Network Classifier. *arXiv preprint arXiv:2202.04962* (2022). https://doi.org/10.48550/arXiv.2202.04962

[16] Yang, B., Li, S., Feng, J. & Vasile, M. Fast Solver for J2-Perturbed Lambert Problem Using Deep Neural Network. *Journal of Guidance, Control, and Dynamics* **45**, 875–884 (2022). https://doi.org/10.2514/1.G006091

[17] Liu, Y., Noomen, R. & Visser, P. A Gravity Assist Mapping Based on Gaussian Process Regression. *The Journal of the Astronautical Sciences* **68**, 248–272 (2021). https://doi.org/10.1007/s40295-021-00246-3

[18] Izzo, D., Sprague, C. & Tailor, D. Machine learning and evolutionary techniques in interplanetary trajectory design. *Modeling and Optimization in Space Engineering*, 191–210 (2019). https://doi.org/10.1007/978-3-030-10501-3_8

[19] Sánchez-Sánchez, C. & Izzo, D. Real-Time Optimal Control via Deep Neural Networks: Study on Landing Problems. *Journal of Guidance, Control, and Dynamics* **41**, 1122–1135 (2018). https://doi.org/10.2514/1.G002357

[20] Cheng, L., Wang, Z., Song, Y. & Jiang, F. Real-time optimal control for irregular asteroid landings using deep neural networks. *Acta Astronautica* **170**, 66–79 (2020). https://doi.org/10.1016/j.actaastro.2019.11.039

[21] Izzo, D. & Öztürk, E. Real-Time Guidance for Low-Thrust Transfers Using Deep Neural Networks. *Journal of Guidance, Control, and Dynamics* **44**, 315–327 (2021). https://doi.org/10.2514/1.G005254

[22] Rubinsztejn, A., Sood, R. & Laipert, F. E. Neural network optimal control in astrodynamics: Application to the missed thrust problem. *Acta Astronautica* **176**, 192–203 (2020). https://doi.org/10.1016/j.actaastro.2020.05.027

[23] Zavoli, A. & Federici, L. Reinforcement Learning for Robust Trajectory Design of Interplanetary Missions. *Journal of Guidance, Control, and Dynamics* **44**, 1440–1453 (2021). https://doi.org/10.2514/1.G005794

[24] Peng, H. & Bai, X. Artificial Neural Network–Based Machine Learning Approach to Improve Orbit Prediction Accuracy. *Journal of Spacecraft and Rockets* **55**, 1248–1260 (2018). https://doi.org/10.2514/1.A34171

[25] Peng, H. & Bai, X. Machine Learning Approach to Improve Satellite Orbit Prediction Accuracy Using Publicly Available Data. *The Journal of the Astronautical Sciences* **67**, 762–793 (2020). https://doi.org/10.1007/s40295-019-00158-3

[26] Caldas, F. & Soares, C. Machine learning in orbit estimation: A survey. *Acta Astronautica* **220**, 97–107 (2024). https://doi.org/10.1016/j.actaastro.2024.03.072

[27] Uriot, T. et al. Spacecraft Collision Avoidance Challenge: design and results of a machine learning competition. *Astrodynamics* **6**, 121–140 (2021). https://doi.org/10.1007/s42064-021-0101-5

[28] Pinto, F. et al. Towards Automated Satellite Conjunction Management with Bayesian Deep Learning. *arXiv preprint arXiv:2012.12450* (2020). https://doi.org/10.48550/arXiv.2012.12450

[29] Acciarini, G. et al. Kessler: A Machine Learning Library for Spacecraft Collision Avoidance. In *8th European Conference on Space Debris* (2021).

[30] Sánchez Fernández-Mellado, L. & Vasile, M. On the use of Machine Learning and Evidence Theory to improve collision risk management. *Acta Astronautica* **181**, 694–706 (2021). https://doi.org/10.1016/j.actaastro.2020.08.004

[31] Stevenson, E., Rodríguez-Fernández, V., Urrutxua, H. & Camacho, D. Benchmarking deep learning approaches for all-vs-all conjunction screening. *Advances in Space Research* **72**, 2660–2675 (2023). https://doi.org/10.1016/j.asr.2023.01.036

[32] Hundman, K., Constantinou, V., Laporte, C., Colwell, I. & Söderström, T. Detecting Spacecraft Anomalies Using LSTMs and Nonparametric Dynamic Thresholding. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 387–395 (2018). https://doi.org/10.1145/3219819.3219845

[33] Tariq, S. et al. Detecting Anomalies in Space using Multivariate Convolutional LSTM with Mixtures of Probabilistic PCA. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2123–2133 (2019). https://doi.org/10.1145/3292500.3330776

[34] Naik, K. R., Holmgren, A. & Kenworthy, J. Using Machine Learning to Automatically Detect Anomalous Spacecraft Behavior from Telemetry Data. In *IEEE Aerospace Conference* (2020). https://doi.org/10.1109/AERO47225.2020.9172726

[35] Jiang, S., Jiang, Y., Wang, Y., Zhang, X. & Zhang, Z. Anomaly Detection in Spacecraft Telemetry Data Based on Transformer-LSTM. In *International Conference on Intelligent Communication and Networking (ICN)* (2023). https://doi.org/10.1109/ICN60549.2023.10426290

[36] Herrmann, L., Bieber, M., Verhagen, W. J. C., Cosson, F. & Santos, B. F. Unmasking overestimation: a re-evaluation of deep anomaly detection in spacecraft telemetry. *CEAS Space Journal* **16**, 225–237 (2024). https://doi.org/10.1007/s12567-023-00529-5

[37] Kotowski, K. et al. European Space Agency Benchmark for Anomaly Detection in Satellite Telemetry. *arXiv preprint arXiv:2406.17826* (2024). https://doi.org/10.48550/arXiv.2406.17826

[38] Ruszczak, B., Kotowski, K., Evans, D. & Nalepa, J. The OPS-SAT benchmark for detecting anomalies in satellite telemetry. *Scientific Data* **12**, 710 (2025). https://doi.org/10.1038/s41597-025-05035-3

[39] Hughes, S. P., Qureshi, R. H., Cooley, D. S. & Parker, J. J. K. Verification and Validation of the General Mission Analysis Tool (GMAT). In *AIAA/AAS Astrodynamics Specialist Conference* (2014). https://doi.org/10.2514/6.2014-4151

[40] Papyan, V., Han, X. Y. & Donoho, D. L. Prevalence of Neural Collapse during the Terminal Phase of Deep Learning Training. *Proceedings of the National Academy of Sciences* **117**, 24652–24663 (2020). https://doi.org/10.1073/pnas.2015509117

[41] Vaswani, A. et al. Attention Is All You Need. In *Advances in Neural Information Processing Systems* **30**, 5998–6008 (2017). https://doi.org/10.48550/arXiv.1706.03762

[42] Chen, T. & Guestrin, C. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794 (2016). https://doi.org/10.1145/2939672.2939785

[43] Saxena, A., Goebel, K., Simon, D. & Eklund, N. Damage Propagation Modeling for Aircraft Engine Run-to-Failure Simulation. In *International Conference on Prognostics and Health Management*, 1–9 (2008). https://doi.org/10.1109/PHM.2008.4711414

[44] Li, X., Ding, Q. & Sun, J.-Q. Remaining Useful Life Estimation in Prognostics Using Deep Convolution Neural Networks. *Reliability Engineering & System Safety* **172**, 1–11 (2018). https://doi.org/10.1016/j.ress.2017.11.021

Funding

The authors received no funding for this work.

Author contributions

Harsha Vardan M Sakamuri: conceptualization, methodology, software, validation, formal analysis, investigation, data curation, writing of the original draft, review and editing, visualization. Rohit Michael Raj: conceptualization, methodology, software, validation,

formal analysis, investigation, data curation, writing of the original draft, review and editing, visualization. Valarmathi P.: supervision.

Competing interests

The authors declare no competing interests.